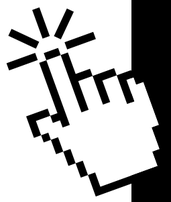
SC2006 - SCSD Group 3

Job Hack SG



Team Members

1	Vivek Shrey
2	Dharshini Saravanan
3	Li Sinuo
4	Sundari
5	Ching Xin Wei

JobHack

Table of Contents



- Our Product
- Product Features
- Software Engineering Practices & Design Patterns
- Good SWE Practices
- Traceability
- Design Patterns
- Future Plans
- Live Demo

JobHack

About JobHack SG



Job Hack SG aims to help students and early-career jobseekers discover where demand is growing in Singapore and which roles match their skills today. Combining sector insights (growth, vacancies, in-demand skills) with a personal matching engine over live job postings, users will be able to make informed real-time decisions to get the best out of their job search journey.

Core Features

- Sector Insights
- Skills Matching
- Profile Specific
- Singapore-centric

Our Goal

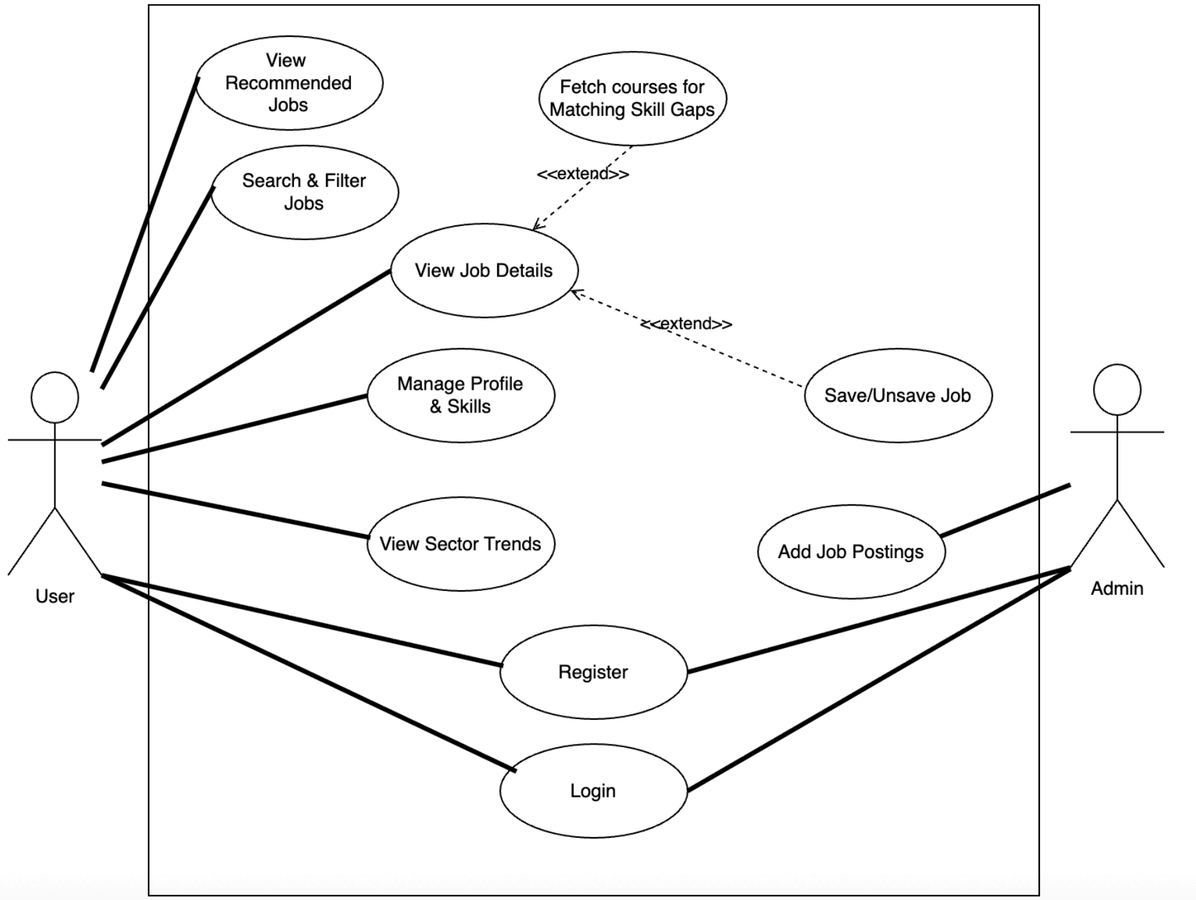
The goal is a clear, testable, end-to-end prototype that demonstrates sector specific guidance along with practical role recommendations, built primarily on public datasets/APIs, specific to the Singapore context.

What sets us apart

What sets Job Hack SG apart from other competing platforms is that this will be a one stop portal for career related matters, and will cater specially for Singapore.

JobHack

Use Case Model



JobHack

Product Features

Authentication
- Registration
- Login
- Admin Sign In

Profile customisation
- Enter Profile Details
- Add / Delete Skills

View Sector Trends
- Display job trends based on vacancies / growth
- Filtered based on User's interested jobs

View Recommended Jobs
- Display jobs that are ideal for users based on their requirements and skills.

Searching / Filtering Jobs
- Filtering function
- Search function based on key words

View Job Details
- Display details like salary, job descriptions
- Save /Unsave Job

Save Job
- Saves job to user's database and can be viewed later.

Fetch Courses for matching Skill gaps
- Displays course links to supplement any missing skills for job role

External APIs



Sector API

- Quarterly data on the number of job vacancies in Singapore.
- Breakdown by industry (based on SSIC 2020 classification) and occupational group.
- Enables analysis of labour demand trends, industry growth, and emerging skills gaps.

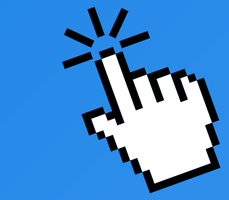
JobHack

Software Engineering Practices & Design Patterns



JobHack

Good SWE Practices





- Documentation
- Good Code Practices
- Reusability & Refactoring
- SCRUM

JobHack

Documentation

JobHack SG

JobHack SG is a comprehensive job discovery and upskilling platform designed specifically for Singaporean job seekers. Built with a modern tech stack (React + TypeScript frontend, Java Spring Boot backend), it leverages real-time data from data.gov.sg to provide personalized job recommendations, skill gap analysis, and upskilling pathways.

🎯 Overview

JobHack SG bridges the gap between job seekers and opportunities by:

- Aggregating job listings with detailed skill requirements
- Recommending personalized upskilling courses based on skill gaps
- Visualizing real-time employment trends across sectors
- Providing a personalized job discovery experience

Whether you're a fresh graduate, career switcher, or professional looking to upskill, JobHack SG helps you navigate Singapore's job market with data-driven insights.

✨ Key Features

Personalized Job Feed

- Browse curated job listings tailored to your skills and preferences
- Filter by industry, experience level, salary range, and location
- View detailed job descriptions with required skills highlighted

Smart Course Recommendations

- Get course suggestions based on your skill gaps
- Discover upskilling pathways to qualify for desired positions
- Access courses from reputable platforms (SkillsFuture, Coursera, Udemy, etc.)

Sector Trends Dashboard

- Visualize job vacancy trends across different industries
- Analyze hiring patterns using live data from data.gov.sg APIs
- Make informed career decisions based on market demand

Saved Jobs & Bookmarks

- Bookmark interesting job listings for later review
- Organize saved jobs by priority or application status

2. Backend Setup (Spring Boot)

Navigate to the backend directory:

```
cd backend
```

Install Dependencies & Run

```
# Compile the project
mvn clean compile

# Run the Spring Boot application
mvn spring-boot:run
```

The backend server will start at `http://localhost:8080`

3. Frontend Setup (React + Vite)

Open a new terminal and navigate to the frontend directory:

```
cd frontend
```

Install Dependencies

```
npm install
```

Run Development Server

```
npm run dev
```

The frontend will be available at `http://localhost:3000` (or the port shown in your terminal)

4. Access the Application

Open your browser and navigate to:

```
http://localhost:3000/onboarding
```

READ ME

A guide for future developers to ensure consistent and reliable updates

Documentation

backend > src > main > java > sc2006 > service > J JobSearchService.java > { } sc2006.service

```java
public class JobSearchService {
    public List<JobPosting> searchJobs(Map<String, Object> filters) {

        for (JobPosting job : allJobs) {
            if (!job.isActive()) continue;

            boolean match = true;

            // Filter by keyword (title, company, or description)
            if (!keyword.isEmpty() && !(job.getTitle().toLowerCase().contains(keyword)
                    || job.getCompany().toLowerCase().contains(keyword)
                    || job.getDescription().toLowerCase().contains(keyword))) {
                match = false;
            }

            // Filter by location
            if (!location.isEmpty() && !job.getLocation().toLowerCase().contains(location)) {
                match = false;
            }

            // Filter by company
            if (!company.isEmpty() && !job.getCompany().toLowerCase().contains(company)) {
                match = false;
            }

            // Filter by posted date
            if (!postedAfter.isEmpty()) {
                LocalDateTime cutoff = getCutoffDate(postedAfter);
                if (job.getPostedAt() != null && job.getPostedAt().isBefore(cutoff)) {
                    match = false;
                }
            }

            if (match) {
```

backend > src > main > java > sc2006 > entity > J JobPosting.java > { } sc2006.entity

```java
/**
 * Represents a job listing.
 */
public class JobPosting {

    private Long jobId;
    private String title;
    private String company;
    private String location;
    private String description;
    private LocalDateTime postedAt;
    private boolean active;
    private URL applyUrl; // optional (if you use external links)
    private String salary; // e.g., "$50k - $80k"
    private String jobType; // "Full-time", "Part-time", "Contract", "Internship"
    private String requiredSkills; // comma-separated skills

    // === Constructors ===
    public JobPosting() {}

    // Minimal constructor (used by DAO)
    public JobPosting(Long jobId, String title, String compan
                      String description, LocalDateTime poste
        this.jobId = jobId;
        this.title = title;
        this.company = company;
        this.location = location;
        this.description = description;
        this.postedAt = postedAt;
        this.active = active;
    }

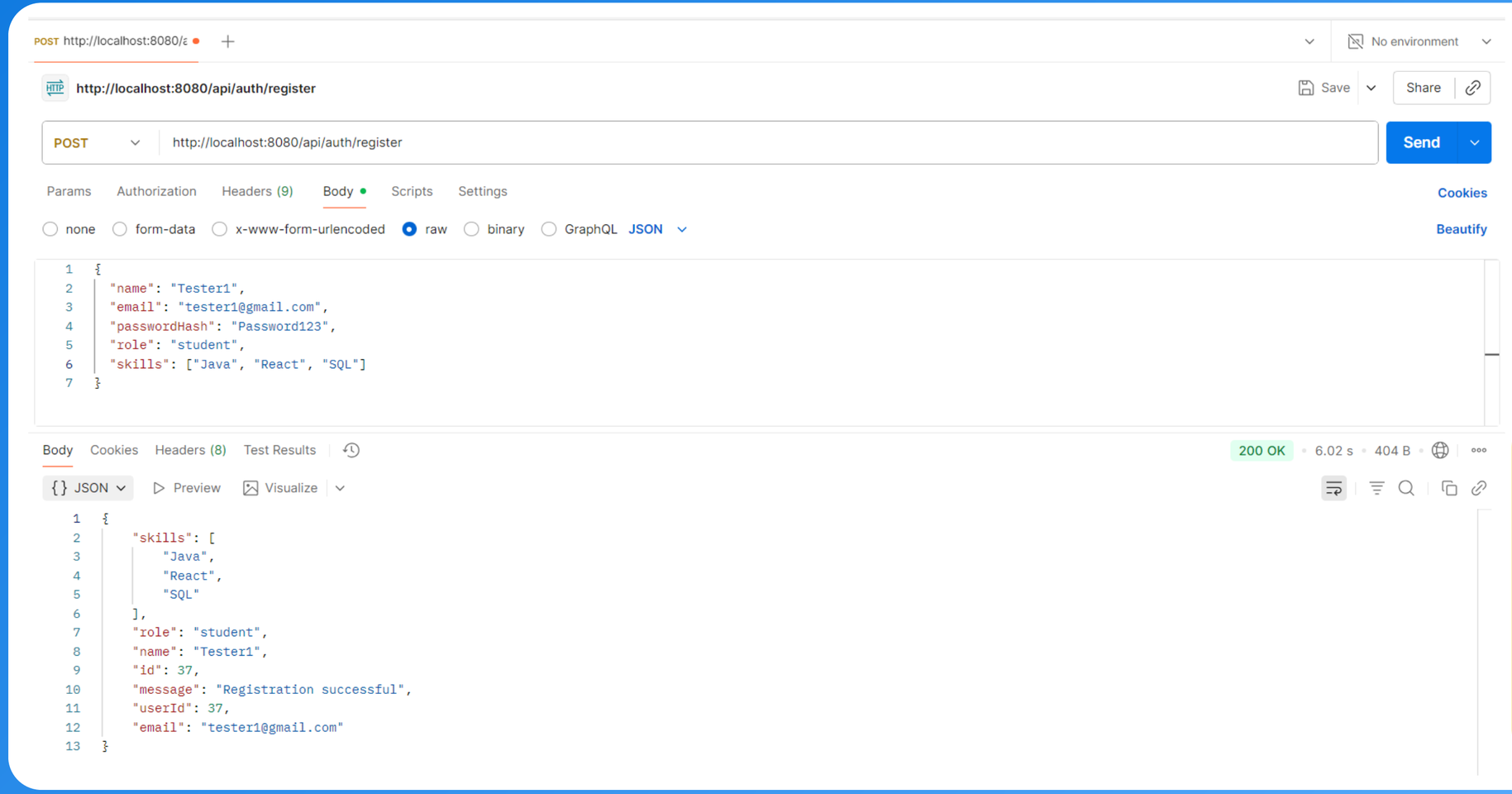
    // Extended constructor with URL (optional)
    public JobPosting(Long jobId, String title, String company,
```

COMMENTS

To help future developers better understand and make sense of the code

JobHack

Documentation



Using Postman to test our backend endpoints ensures both frontend and backend teams can collaborate seamlessly and verify real-time data flow.

API Docs

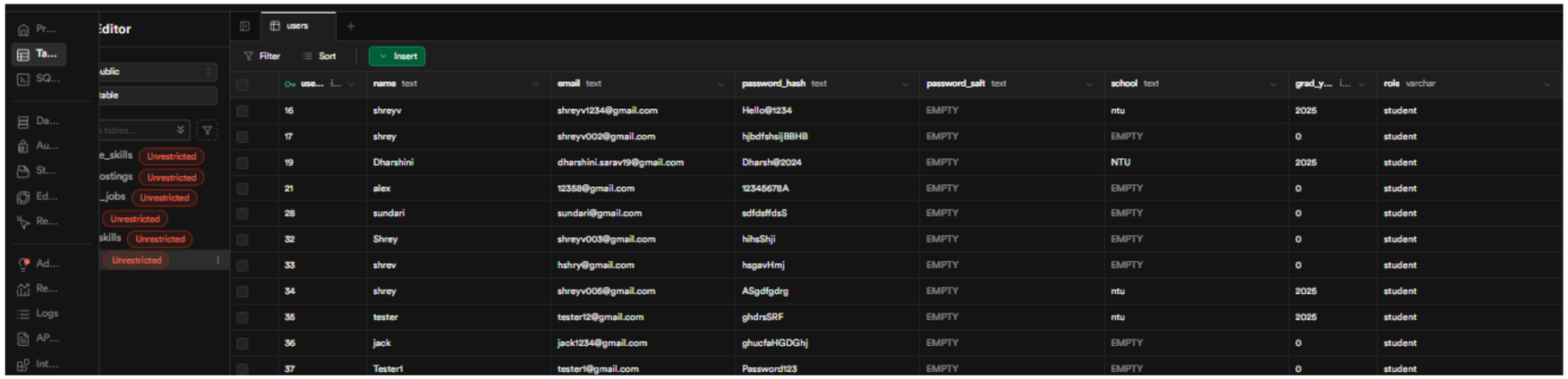
To let the Frontend & Backend teams collaborate with one another and allow future extensibility

JobHack

Documentation

```
2025-11-03T18:54:36.433+08:00  INFO 7028 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet        : Completed initialization in 1 ms
[Register] Received userData: {name=Tester1, email=tester1@gmail.com, passwordHash=Password123, role=student, skills=[Java, React, SQL]}
[Register] Creating user: sc2006.entity.User@2d63ccd2
Hibernate: select u1_0.user_id,u1_0.email,u1_0.grad_year,u1_0.name,u1_0.password_hash,u1_0.password_salt,u1_0.role,u1_0.school from users u1_0 where u1_0.email=?
Hibernate: insert into users (email,grad_year,name,password_hash,password_salt,role,school) values (?,?,?,?,?,?,?)
[Register] User saved: sc2006.entity.User@2d63ccd2
[Register] Added skills: [Java, React, SQL]
[Register] Registration successful for userId: 37
```

Spring Boot backend logs confirming successful request processing



user_id	name	email	password_hash	password_salt	school	grad_year	role
16	shreyv	shreyv1234@gmail.com	Hello@1234	EMPTY	ntu	2025	student
17	shrey	shreyv002@gmail.com	hjbdfshsjBBHB	EMPTY	EMPTY	0	student
19	Dharshini	dharshini.sarav19@gmail.com	Dharsh@2024	EMPTY	NTU	2025	student
21	alex	12358@gmail.com	12345678A	EMPTY	EMPTY	0	student
28	sundari	sundari@gmail.com	sdfdsffdsS	EMPTY	EMPTY	0	student
32	Shrey	shreyv003@gmail.com	hihsShji	EMPTY	EMPTY	0	student
33	shrev	hshry@gmail.com	hsgavHmj	EMPTY	EMPTY	0	student
34	shrey	shreyv005@gmail.com	ASgdfgdrg	EMPTY	ntu	2025	student
35	tester	tester12@gmail.com	ghdrsSRF	EMPTY	ntu	2025	student
36	jack	jack1234@gmail.com	ghucfaHGDGhj	EMPTY	EMPTY	0	student
37	Tester1	tester1@gmail.com	Password123	EMPTY	EMPTY	0	student

Database updated automatically via API request, ensuring consistent data between client and server.

API Docs

To let the Frontend & Backend teams collaborate with one another and allow future extensibility

Documentation



```typescript
const handleSkillClick = (skill: string) => {
  if (selectedSkills.includes(skill)) {
    setSelectedSkills(selectedSkills.filter((s) => s !== skill));
  } else {
    setSelectedSkills([...selectedSkills, skill]);
  }
};

const addCustomSkill = () => {
  if (customSkill.trim() && !selectedSkills.includes(customSkill)) {
    setSelectedSkills([...selectedSkills, customSkill.trim()]);
    setCustomSkill("");
  }
};

const validateEmail = (email: string): string | null => {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailRegex.test(email)) {
    return "Please enter a valid email address";
  }
  return null;
};
```

We maintained consistent naming conventions across our frontend — using camelCase for functions and descriptive names to improve readability and maintainability

```java
@PostMapping("/{userId}/skills")
public Map<String, Object> updateUserSkills(@PathVariable Long userId, @RequestBody Map<String, Object> skillsData) {
    Map<String, Object> response = new HashMap<>();

    try {
        @SuppressWarnings("unchecked")
        java.util.List<String> skills = (java.util.List<String>) skillsData.get("skills");
        userProfileService.updateUserSkills(userId, skills);
        response.put("success", true);
        response.put("message", "Skills updated successfully");
    } catch (Exception e) {
        response.put("success", false);
        response.put("error", e.getMessage());
    }

    return response;
}
```

Our backend followed Java naming standards and structured response formats for clarity and consistency with frontend expectations.

Code Consistency

Maintains code style & consistency to ensure readability and maintenance of code

Reusability & Refactoring

Removal of Duplicate Code

```
const validateEmail = (email: string): string | null => {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailRegex.test(email)) {
    return "Please enter a valid email address";
  }
  return null;
};
```

Centralized validation logic to prevent repetition.

Removal of Redundant objects

Body | Cookies | Headers (8) | Test Results

{} JSON | Preview | Visualize


```
{
    "skills": [
        "Java",
        "React",
        "SQL"
    ],
    "role": "student",
    "name": "Tester1",
    "id": 37,
    "message": "Registration successful",
    "userId": 37,
    "email": "tester1@gmail.com"
}
```


Removed unnecessary fields in backend responses.

Reusability & Refactoring

Shortening Methods

```
if (!isLogin) {
  const emailValidationError = validateEmail(email);
  if (emailValidationError) {
    setEmailError(emailValidationError);
    return;
  }

  const passwordValidationError = validatePassword(password);
  if (passwordValidationError) {
    setPasswordError(passwordValidationError);
    return;
  }
}
```

Refactored long methods into reusable helpers.

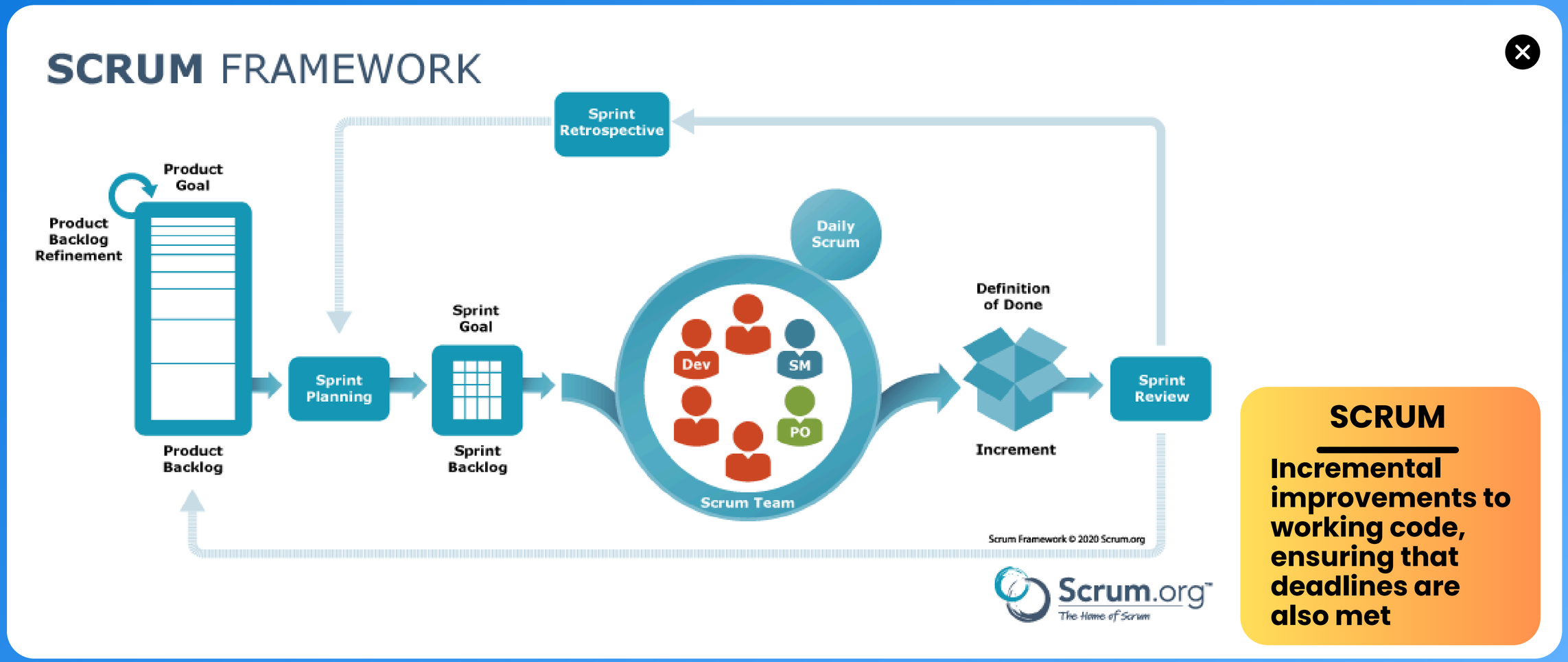
Loose coupling

```
  const res = await fetch(endpoint, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(payload),
  });
```

Frontend and backend are decoupled via REST API.

JobHack

SCRUM



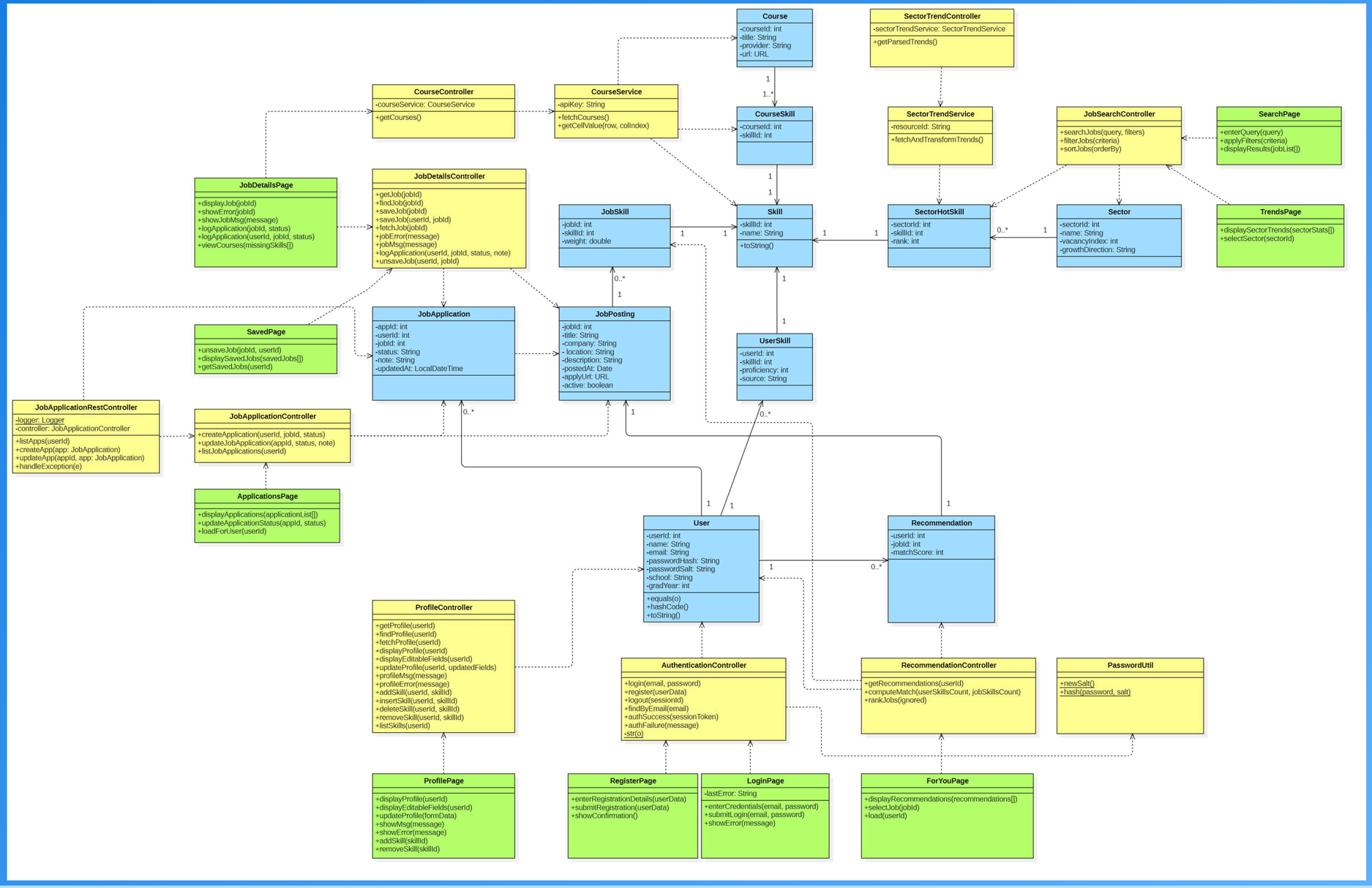
SCRUM

Incremental improvements to working code, ensuring that deadlines are also met

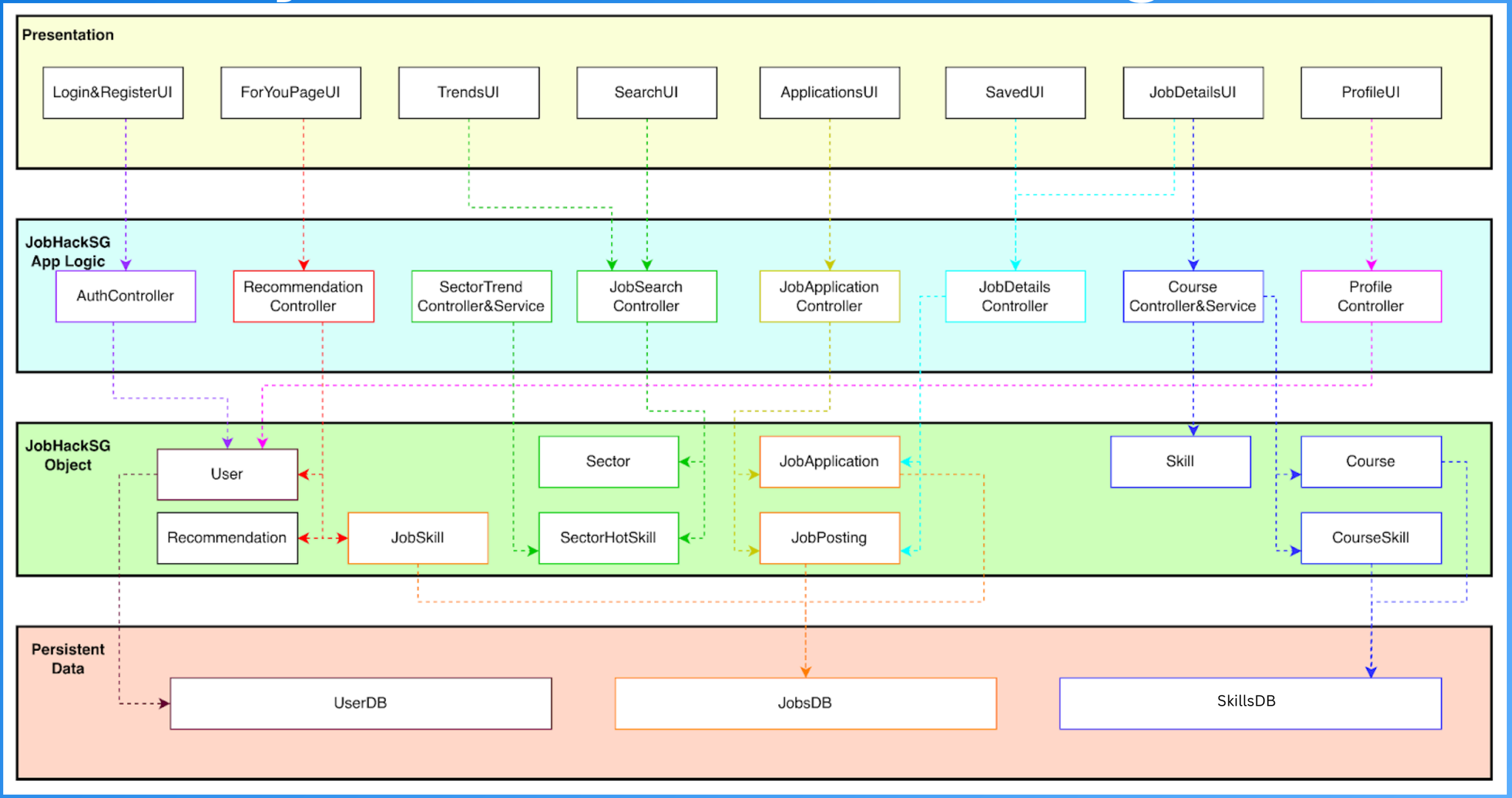
JobHack

Class Diagram





System Architecture Diagram



Tech Stack



Frontend:

- Framework - React + Typescript
- Styling - TailwindCSS + shadcn/ui
- Icons - lucide-react
- Build Tool - Vite

Backend:

- Language - Java (Spring Boot)
- Database - Supabase
- API Integration - data.gov.sg APIs
- Server Tools - Maven (for Spring Boot) / npm (for Node.js)

JobHack

Design Patterns



MVC Design Pattern



Model

The data layer manages data, logic, and rules of the application.

In our code, Java classes like Course, Job, SectorTrend, and User represent data entities; CourseService, JobService, handle fetching and processing data (e.g. from APIs or databases).

View

The presentation layer displays data to users.

The React frontend components (e.g. JobList.tsx, JobDetails.tsx, CourseSuggestions.tsx) render data received from the backend API.

Controller

The request handler processes user input, updates the model, and refreshes the view.

Spring Boot controllers like JobController, CourseController, or Node.js route handlers in routes/jobRoutes.js handle HTTP requests and call the appropriate services.

Other Patterns



Observer Pattern

The state management system in React (useState, useEffect, context) follows the Observer pattern. Components observe changes in state and automatically re-render when the observed data changes.

This pattern:

- Promotes reactive UI updates.
- Makes the View automatically reflect Model changes without manual DOM manipulation.

SOLID PRINCIPLES

Single Responsibility Principle (SRP)

Each class is only in charge of one function. Each controller/page focuses on one area (e.g.,ProfileController, JobApplicationController)..

Open-Closed Principle (OCP)

A class will not be modified for future extensions. For instance, no new attributes will be added to “Skill” even if new skill types are added.



JobHack

SOLID PRINCIPLES



Liskov Substitution Principle (LSP)

if in the future *JobApplication* had subclasses like *InternshipApplication* or *FullTimeApplication*, these objects can be passed into all the methods of the controller.

Dependency Inversion Principle (DIP)

Pages (UI) depend on controllers, not on models directly.
For example, ApplicationsPage depends on JobApplicationController, not JobApplication itself

JobHack

Traceability in Project Deliverables



JobHack

Login - Use Case

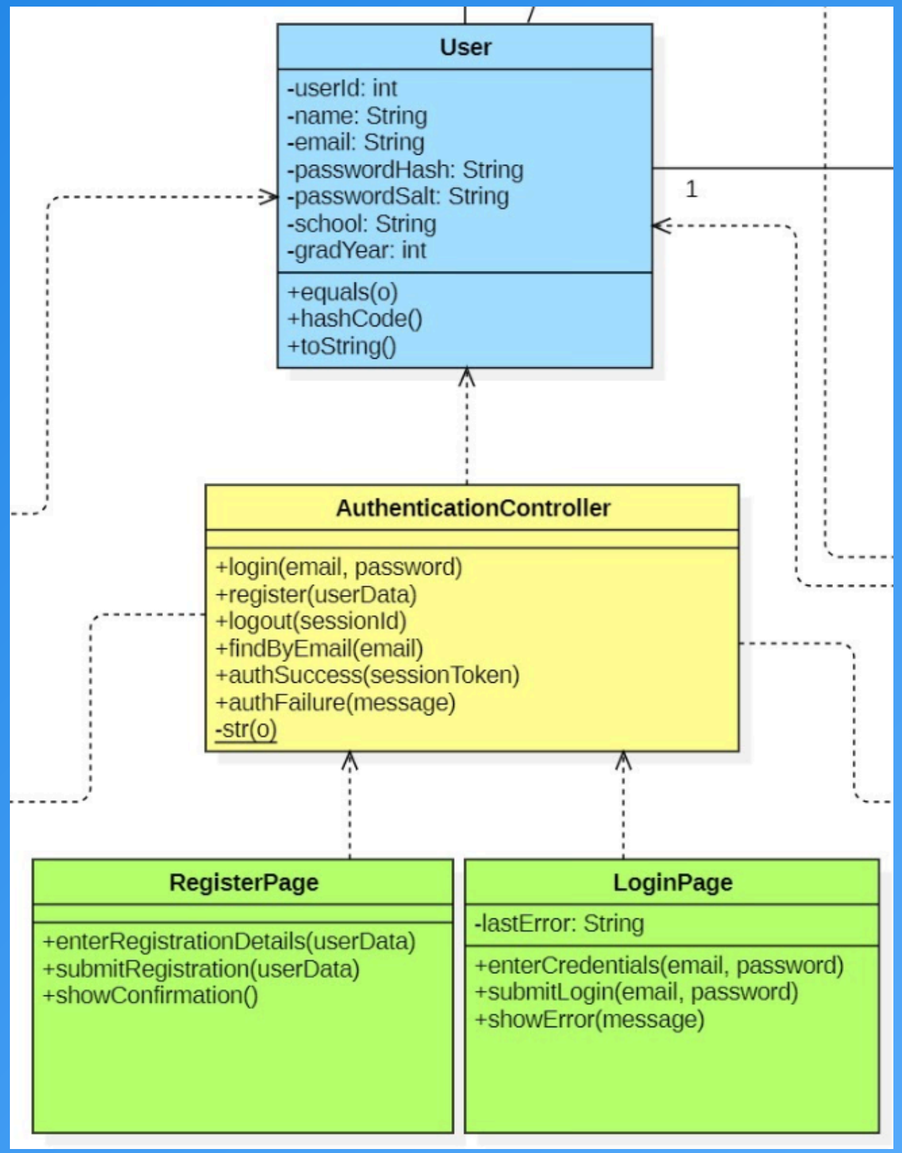


Use Case ID:	**UC-2**		
Use Case Name:	**Login**		
Created By:	Vivek Shrey	Last Updated By:	Vivek Shrey
Date Created:	01 - 09 - 2025	Date Last Updated:	12 - 09 - 2025

Actor:	User
Description:	This is practically the first step for an existing user to enter the application.
Preconditions:	The user already has an existing account, and has been authenticated previously.
Postconditions:	User has logged into their account OR User is given a valid explanation for why their login failed.
Priority:	High
Frequency of Use:	High
Flow of Events:	1. System displays fields for email and password 2. User enters their details into the field. 3. User presses the login button. 4. System verifies their email and password with the account information in the system database. 5. If the account is verified, system displays a message that says 'Login is successful!' 6. User is brought to the home page / landing screen.
Alternative Flows:	**AF-S1:** If the email or password is entered incorrectly 1. The system displays a message saying 'Invalid email or password! Please Try Again!"
	2. Return to Step 3. **AF-S2:** If User leaves the field blank 1. System highlights missing fields with error message "This field is required"
Exceptions:	**EX-1:** System cannot reach database → Error: "Login service temporarily unavailable". **EX-2:** Network disconnected → Error: "Please check your internet connection".
Includes:	Included by UC-1.
Special Requirements:	Passwords must be stored with secure hashing + salting
Assumptions:	User has access to their own email
Notes and Issues:	None

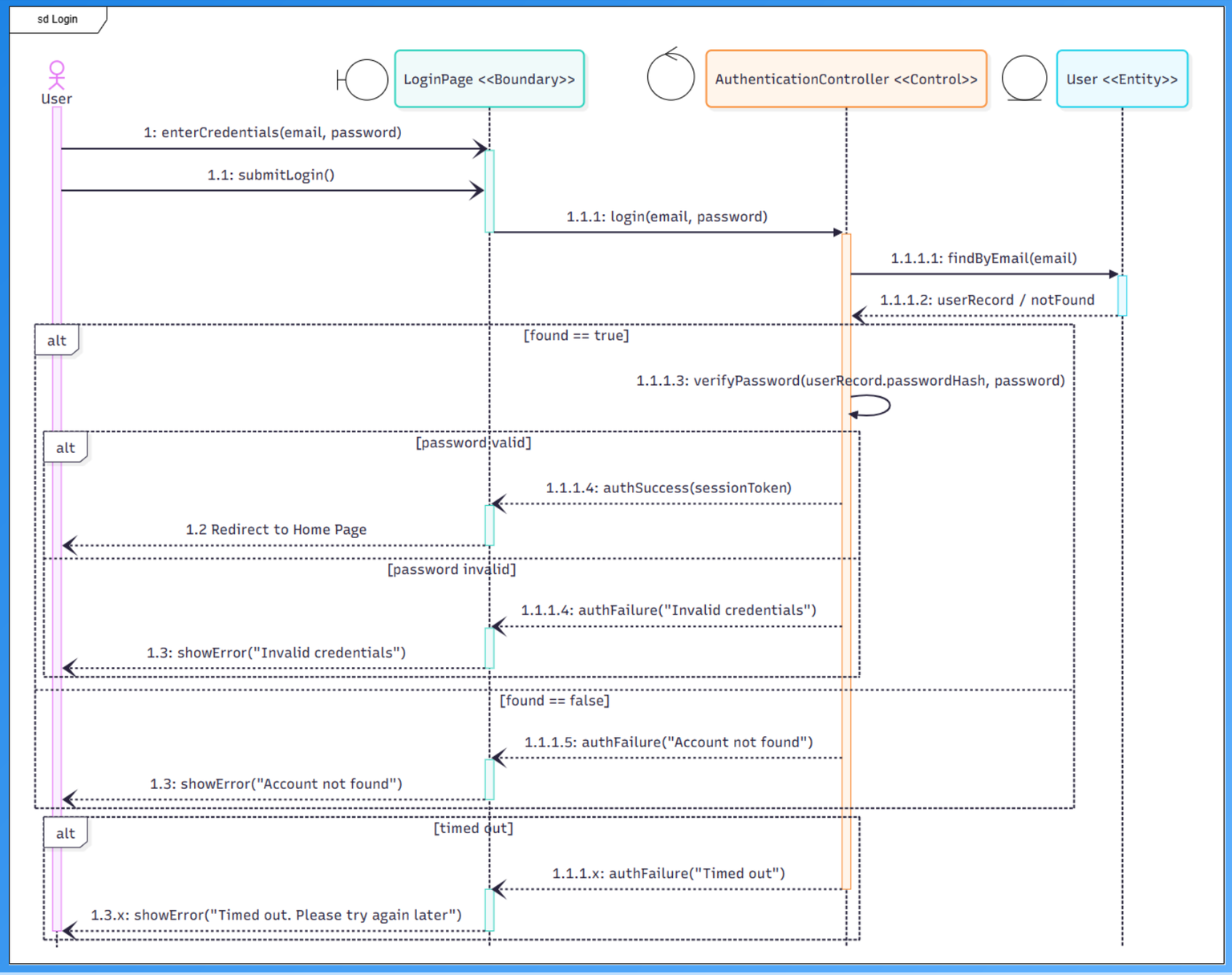
JobHack

Login - Class Diagram



JobHack

Login - Sequence Diagram



Good Designs Applied



```java
// === Login user ===
@PostMapping("/login")
public Map<String, Object> login(@RequestBody Map<String, String> body) {
    String email = body.get("email");
    String password = body.get("passwordHash");

    Map<String, Object> response = new HashMap<>();
    try {
        User user = authService.loginAndReturnUser(email, password);
        if (user == null) {
            response.put("error", "Invalid email or password");
            return response;
        }
        response.put("id", user.getUserId());
        response.put("userId", user.getUserId());
        response.put("name", user.getName());
        response.put("email", user.getEmail());
        response.put("role", user.getRole());
        // Fetch skills for user
        List<String> skills = authService.getUserSkills(user.getUserId());
        response.put("skills", skills);
        response.put("message", "Welcome back! Login successful");
        return response;
    } catch (Exception e) {
        response.put("error", "Something went wrong during login. Please try again later.");
        return response;
    }
}

// === Get user details ===
@GetMapping("/{email}")
public User getUser(@PathVariable String email) {
    return authService.getUser(email);
}
}
```

```java
public class LoginPage {

    public Map<String, Object> submitLogin(String email, String password) {
        try {
            AuthenticationController authController = new AuthenticationController();

            // Build request body in the expected format
            Map<String, String> loginData = new HashMap<>();
            loginData.put("email", email);
            loginData.put("passwordHash", password);

            // Controller returns JSON-like response
            Map<String, Object> response = authController.login(loginData);

            if (response.containsKey("error")) {
                showError((String) response.get("error"));
            }

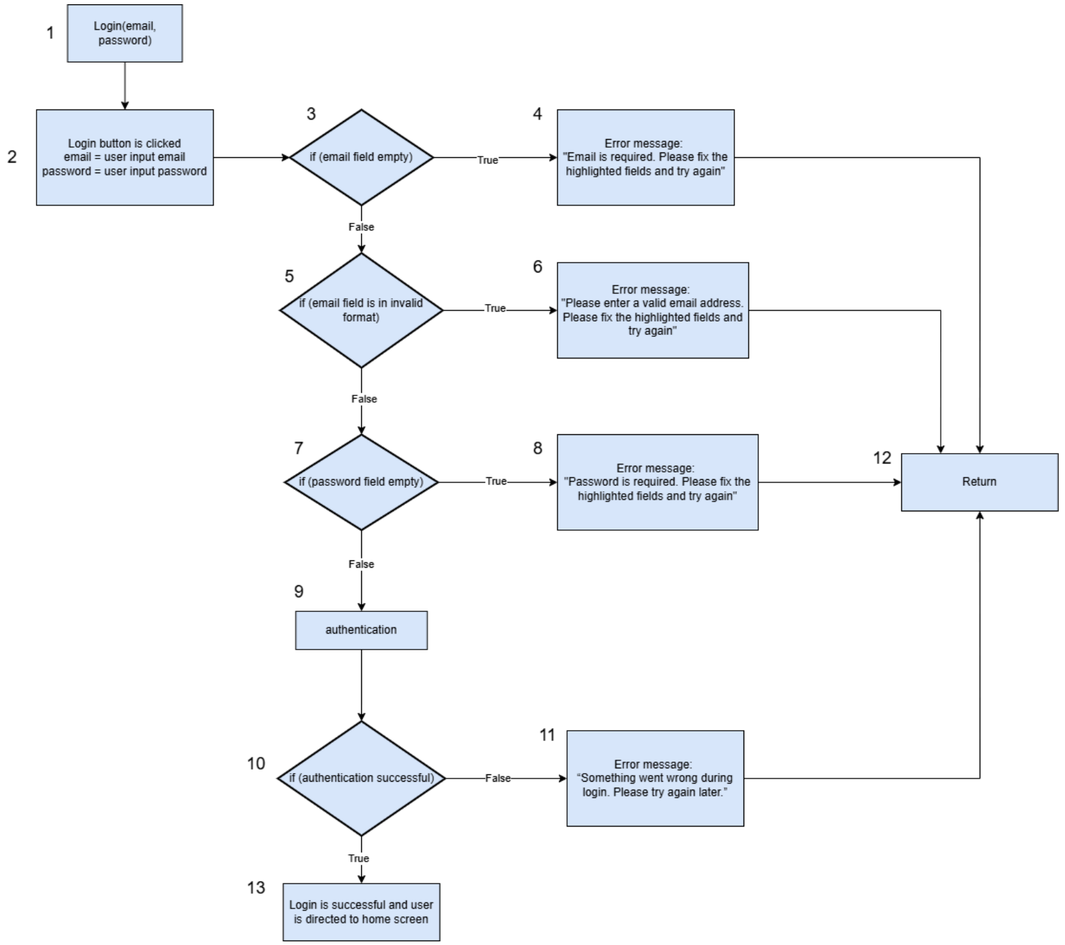
            return response;

        } catch (Exceptions.Auth e) {
            showError(e.getMessage());
            Map<String, Object> err = new HashMap<>();
            err.put("error", e.getMessage());
            return err;
        }
    }

    public void showError(String message) {
        this.lastError = message;
    }

    public String getLastError() {
        return lastError;
    }
```

Testing - Control Flow



Black and White Box Testing

Testing - Basis Path Testing

Basic Path Testing

Cyclomatic Complexity: |decision points| + 1 = 4 + 1 = 5

Basis Paths:

1. Baseline path: 1, 2, 3, 5, 7, 9, 10, 13
2. Basis path #2: 1, 2, 3, 4, 12
3. Basis path #3: 1, 2, 3, 5, 6, 12
4. Basis path #4: 1, 2, 3, 5, 7, 8, 12
5. Basis path #5: 1, 2, 3, 5, 7, 9, 10, 11, 12

Black and White Box Testing

Testing Table

No.	Test Input	Expected Output	Actual Output	Pass?
1	Email = “test123@gmail.com” Password = “Password123”	Successful login	Successful login	Yes
2	Email = “” Password = “Password123”	Login failed, system notify “Email is required. Please fix the highlighted fields and try again”	Login failed, system notify “Email is required. Please fix the highlighted fields and try again”	Yes
3	Email = “test123” Password = “Password123”	Login failed, system notify “Please enter a valid email address. Please fix the highlighted fields and try again”	Login failed, system notify “Please enter a valid email address. Please fix the highlighted fields and try again”	Yes
4	Email = “test123@” Password = “Password123”	Login failed, system notify “Please enter a valid email address. Please fix the highlighted fields and try again”	Login failed, system notify “Please enter a valid email address. Please fix the highlighted fields and try again”	Yes
5	Email = “test123@gmail.com” Password = “”	Login failed, system notify “Password is required. Please fix the highlighted fields and try again”	Login failed, system notify “Password is required. Please fix the highlighted fields and try again”	Yes
6	Email = “test123@gmail.com” Password = “WrongPass222”	Login failed, system notified “Something went wrong during login. Please try again later.”	Login failed, system notified “Something went wrong during login. Please try again later.”	Yes
7	Email = “wrong123@gmail.com”	Login failed, system notified “Something	Login failed, system notified “Something	Yes
	Password = “Password123”	went wrong during login. Please try again later.”	went wrong during login. Please try again later.”	

Black and White Box Testing

Future Plans



Application Notes

Besides just redirecting the user to the application page, the application can also be recorded along with any notes



More Sector Data

The sector trends page can display more specific data such as vacancies to better inform our users



Mobile application

In the future, we can have a mobile application to increase outreach and use of our product.

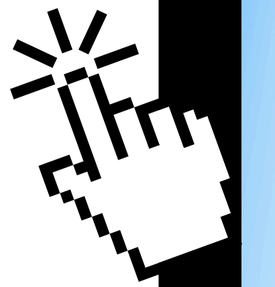


JobHack

Live Demo

JobHack

Thank You



Any Questions?